

Coin Detection

Name: VEDHA M **Reg. No:** 212225230292 **Slot** OD1199

Coin Detection using Morphological Operations and Thresholding

Aim

In this project, you will work with an image and apply morphological operations and thresholding techniques to detect and count the total number of coins present in the image.

Note:

- **Do not remove intermediate images.**
 - Display all key intermediate images (grayscale, thresholded, binary, morphological result, final detections).
 - Document your findings and observations for each step.
 - Print the total number of coins detected using both **blob detection** and **contour detection** methods.
-

Expected Output

All expected output images and results are already displayed.
Enter your code only inside the blocks marked as:

Your Code Here

```
In [5]: import cv2
import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline
```

Step 1: Read Image

```
In [7]: # Read image
# Store it in the variable image
image = cv2.imread("CoinsA.png")
```

```
In [29]: # Dont Change the Code
imageCopy = image.copy()
plt.imshow(image[:, :, :-1]);
plt.title("Original Image")
```

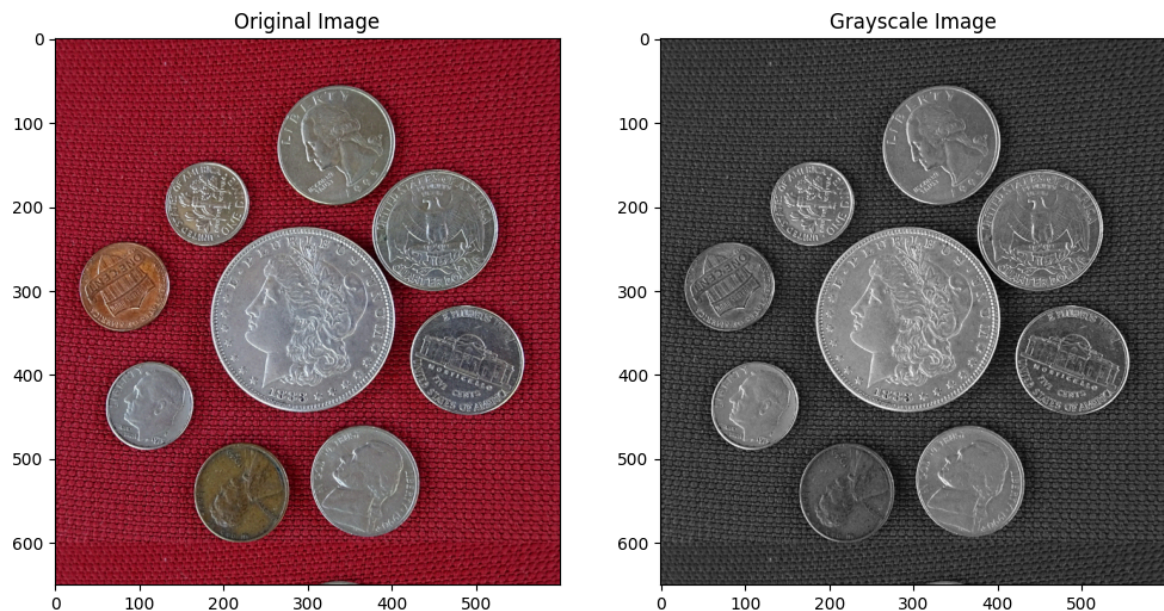
```
plt.show()  
# Expected output
```



Step 2: Convert Image to Grayscale

```
In [8]: # Convert image to grayscale  
# Store it in the variable imageGray  
# Convert image to grayscale  
# Store it in the variable imageGray  
  
imageGray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

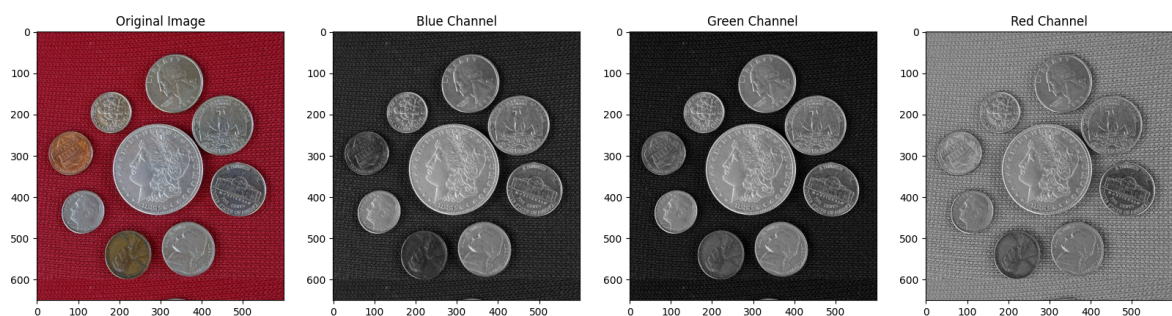
```
In [30]: plt.figure(figsize=(12,12))  
plt.subplot(121);plt.imshow(image[:,:,:-1]);plt.title("Original Image")  
plt.subplot(122); plt.imshow(imageGray,cmap='gray');plt.title("Grayscale Image")  
# Expected output
```



Step 3: Split Image into R,G,B Channels

```
In [10]: # Split cell into channels
# Store them in variables imageB, imageG, imageR
###
imageB,imageG,imageR=cv2.split(image)
###
```

```
In [31]: plt.figure(figsize=(20,12))
plt.subplot(141);plt.imshow(image[:,:,:-1]);plt.title("Original Image")
plt.subplot(142);plt.imshow(imageB,cmap='gray');plt.title("Blue Channel")
plt.subplot(143);plt.imshow(imageG,cmap='gray');plt.title("Green Channel")
plt.subplot(144);plt.imshow(imageR,cmap='gray');plt.title("Red Channel");
plt.show()
# Expected output
```



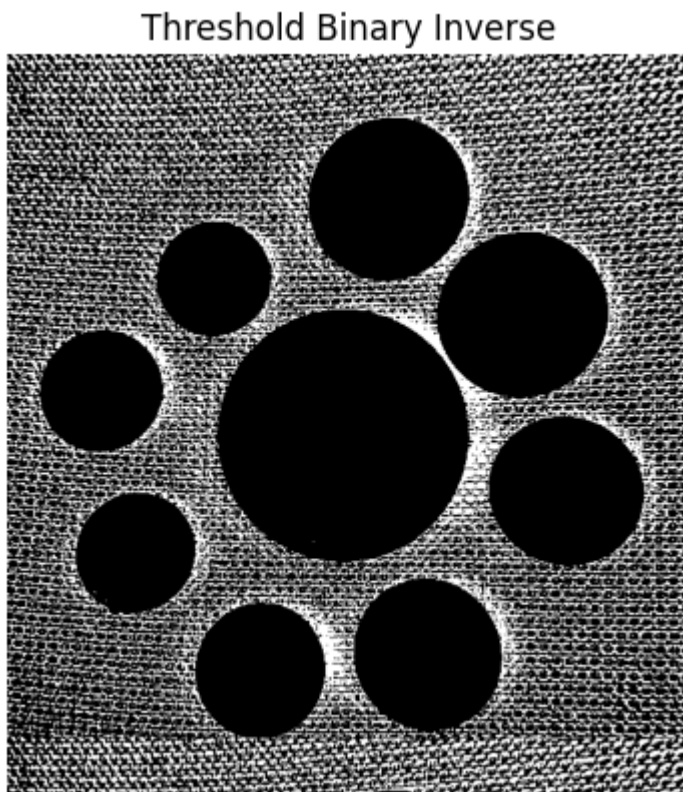
Step 4: Perform Thresholding

You will have to carry out this step with different threshold values to see which one suits you the most. Do not remove those intermediate images and make sure to document your findings.

```
In [11]: ###
thresh=20
maxValue=225
###
```

```
In [12]: # Display the thresholded image
#####
th, dst_bin = cv2.threshold(imageG, thresh, maxValue, cv2.THRESH_BINARY)
th, dst_bin_inv = cv2.threshold(imageG, thresh, maxValue, cv2.THRESH_BINARY_INV)

plt.imshow(dst_bin_inv, cmap='gray')
plt.title('Threshold Binary Inverse')
plt.axis('off')
plt.show()
#####
# Expected output
```



Step 5: Perform morphological operations

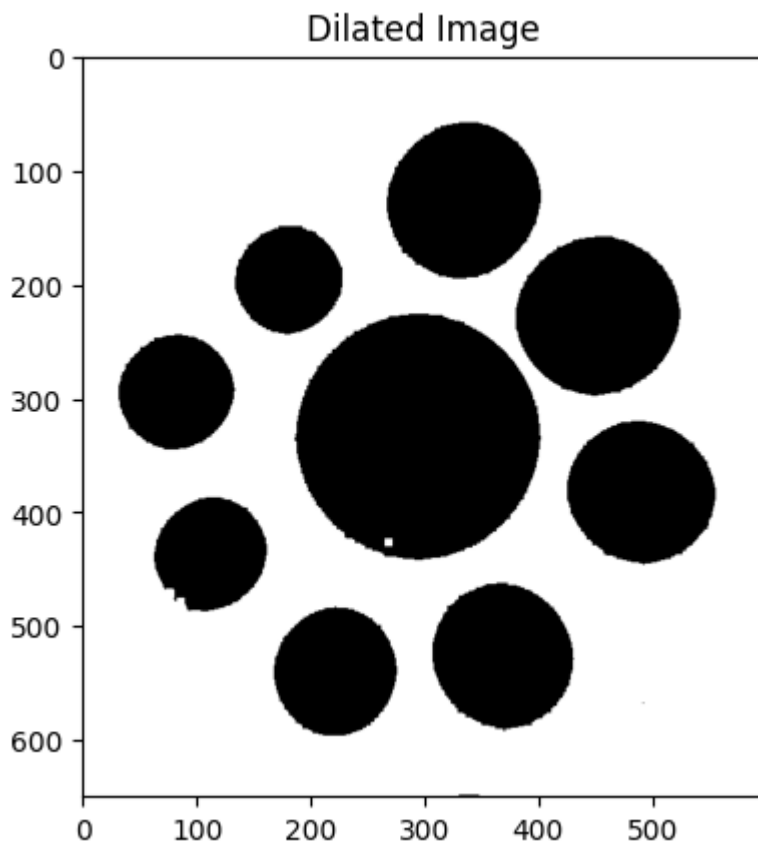
You will have to carry out this step with different kernel size, kernel shape and morphological operations to see which one (or more) suits you the most. Do not remove those intermediate images and make sure to document your findings.

```
In [14]: ###
          kernel=np.ones((7,7),dtype=np.uint8)
          kernel
          ##
```

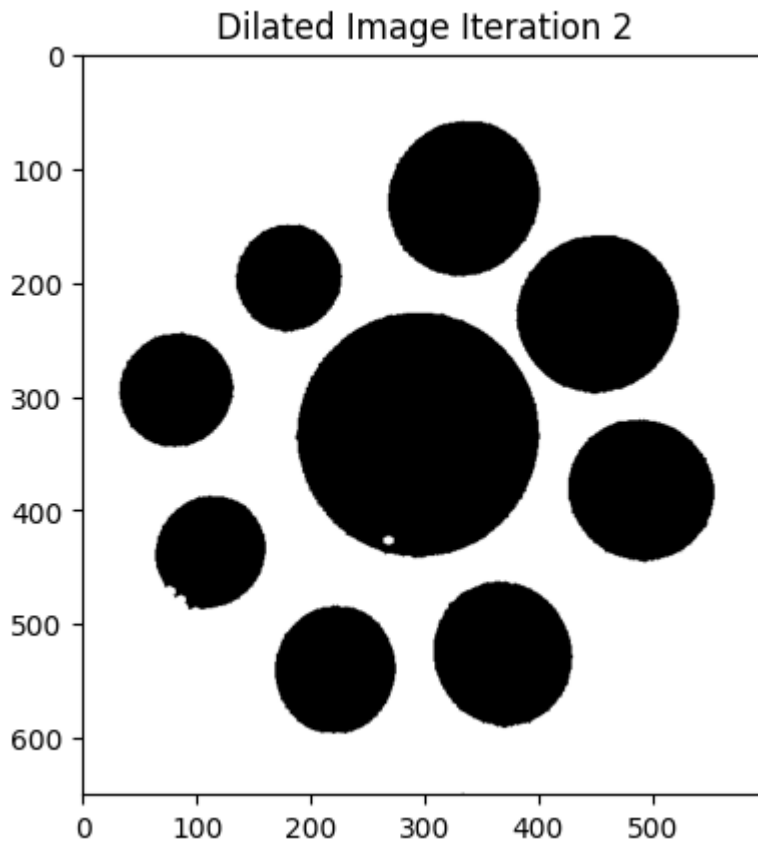
[illegible]

```
In [15]: ###
dst=cv2.dilate(dst_bin_inv,kernel)
###
```

```
In [17]: plt.imshow(dst,cmap='gray');plt.title('Dilated Image');plt.show()
```



```
In [33]: ###
### YOUR CODE HERE
###
plt.imshow(imageDilated2,cmap='gray');plt.title('Dilated Image Iteration 2');plt
# Expected output
```

```
In [18]: # Display all the images
# you have obtained in the intermediate steps
###
plt.figure(figsize=(15, 10))

plt.subplot(231)
plt.imshow(image[:, :, ::-1])
plt.title("Original Image")
plt.axis("off")

plt.subplot(232)
plt.imshow(imageGray, cmap='gray')
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(233)
plt.imshow(dst_bin_inv, cmap='gray')
plt.title("Thresholded Image")
plt.axis("off")

plt.subplot(234)
plt.imshow(dst, cmap='gray')
plt.title("Dilated Image")
plt.axis("off")

plt.subplot(235)
plt.imshow(dst, cmap='gray')
plt.title("Dilated Image Iteration 2")
plt.axis("off")

plt.show()
###
```

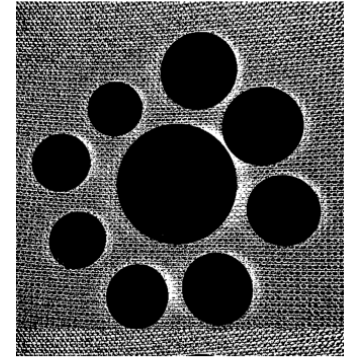
Original Image



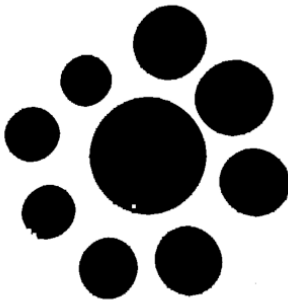
Grayscale Image



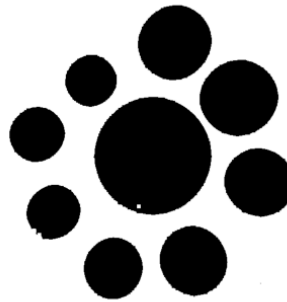
Thresholded Image



Dilated Image

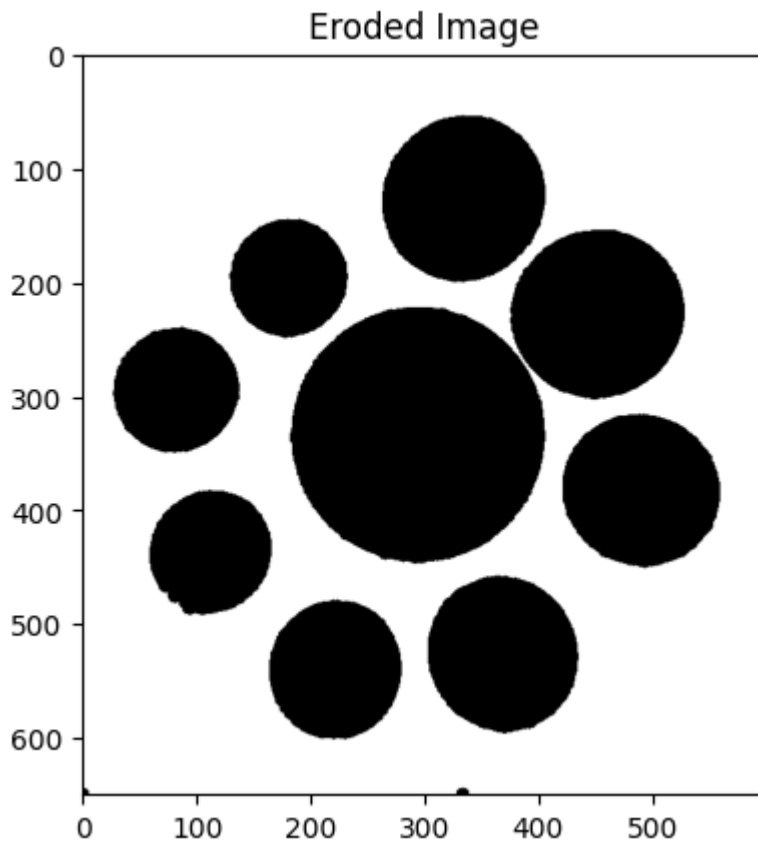


Dilated Image Iteration 2



```
In [19]: # Get structuring element/kernel which will be used for dilation
        ###
        ksize=(7,7)
        k1=cv2.getStructuringElement(cv2.MORPH_ELLIPSE,ksize)
        erod=cv2.erode(dst,k1)
        ###
```

```
In [36]: ###
        ### YOUR CODE HERE
        ###
        plt.imshow(imageEroded,cmap='gray');plt.title("Eroded Image");plt.show()
        # Expected output
```



Step 5: Create SimpleBlobDetector

```
In [22]: # Set up the SimpleBlobdetector with default parameters.
params = cv2.SimpleBlobDetector_Params()

params.blobColor = 0

params.minDistBetweenBlobs = 2

# Filter by Area.
params.filterByArea = False

# Filter by Circularity
params.filterByCircularity = True
params.minCircularity = 0.8

# Filter by Convexity
params.filterByConvexity = True
params.minConvexity = 0.8

# Filter by Inertia
params.filterByInertia = True
params.minInertiaRatio = 0.8
```

```
In [23]: # Create SimpleBlobDetector
detector = cv2.SimpleBlobDetector_create(params)
```

Step 6: Detect Coins

Hints

Use `detector.detect(image)` to detect the blobs (coins). The output of the function is a list of **keypoints** where each keypoint is unique for each blob.

Print the number of coins detected as well.

```
In [28]: # Detect blobs
        ###
        ### YOUR CODE HERE
        ###
        keypoints = detector.detect(erod)
```

```
In [24]: print(f"Number of coins detected: {len(keypoints)}")
        # Expected output
```

Number of coins detected: 9

```
In [30]: # Create a copy of original image
        finalImage = image.copy()

        # Draw circles and centers
        for keypoint in keypoints:

            x, y = keypoint.pt
            radius = keypoint.size / 2

            # Green circle
            cv2.circle(
                finalImage,
                (int(x), int(y)),
                int(radius),
                (0, 255, 0),
                2
            )

            # Blue center
            cv2.circle(
                finalImage,
                (int(x), int(y)),
                4,
                (255, 0, 0),
                -1
            )

        # Convert for matplotlib
        finalImageRGB = cv2.cvtColor(finalImage, cv2.COLOR_BGR2RGB)

        # Show result
        plt.figure(figsize=(8, 8))
        plt.imshow(finalImageRGB)
        plt.title("Final Image")
        plt.axis("off")
        plt.show()
```

Final Image



In []: